

SALESTORM — Scalability & Reliability Design

1. Purpose

SALESTORM is designed for a high-volume flash-sale environment where a sudden traffic spike can cause thousands of customers to request the same limited-stock product simultaneously.

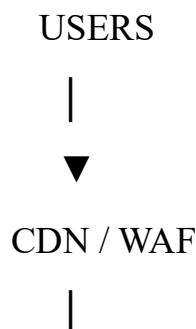
The architecture must support:

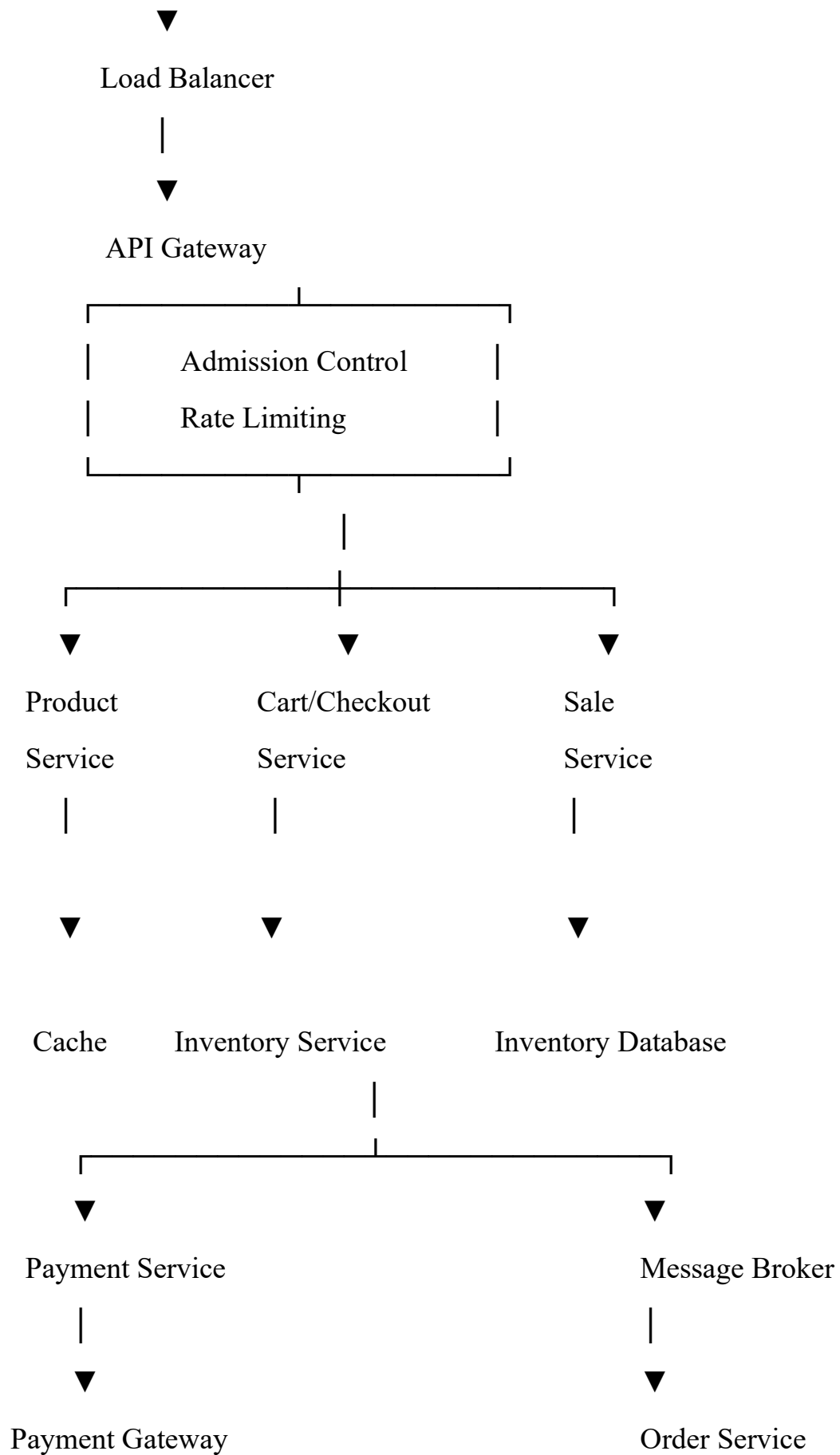
- * 10,000 concurrent purchase requests
- * Approximately 10,000 requests/sec normal traffic
- * Architectural scalability toward 500,000 requests/sec
- * Only 100 available units of the flash-sale product
- * Zero inventory overselling
- * Duplicate-request protection
- * Reliable payment processing
- * Order recovery during service failures
- * Automatic reservation expiry and stock release
- * Horizontal scaling without losing correctness

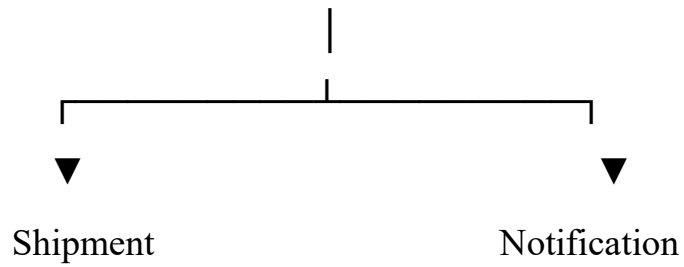
The design separates scalability mechanisms from consistency-critical operations so that increasing traffic does not compromise inventory correctness.

2. Scalability Strategy

The overall scalability architecture is:







3. Horizontal Scaling

SALESTORM keeps application services stateless wherever possible.

Instead of running one server:

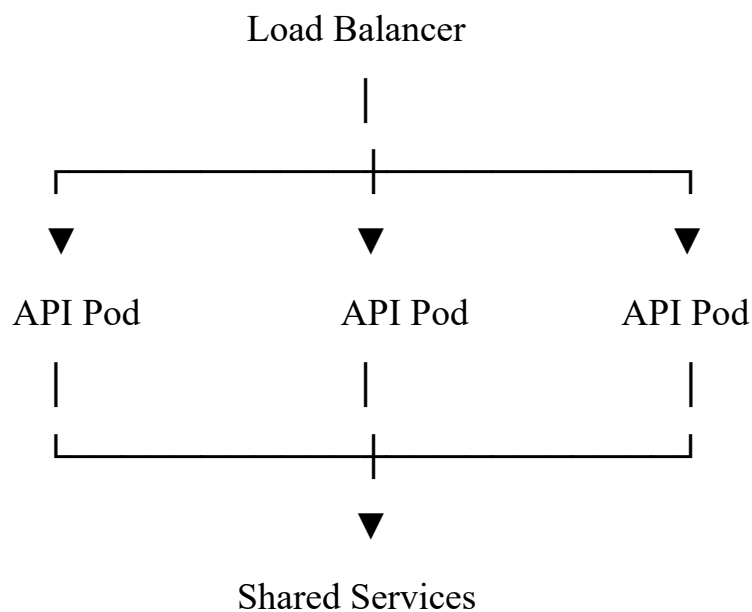
id="h3qvib"

Client



Single Server multiple instances are deployed:

id="6l7p8x"



Benefits:

- * Increased request capacity
- * No single application-server dependency
- * Automatic replacement of failed instances

- * Better fault isolation
- * Supports traffic spikes

Scaling Rule:

When CPU utilization, request rate or latency crosses defined thresholds, additional service instances can be created.

4. API Gateway Admission Control

The API Gateway is the first major traffic-control point.

During a flash sale:

id="7v6q2u"

10,000+ requests



API Gateway



Admission Control



Healthy traffic



Backend Services

The gateway provides:

- * Rate limiting
- * Request authentication
- * Request validation
- * Routing
- * Traffic shaping
- * Burst control

* Protection against abusive clients

Token Bucket:

A token-bucket mechanism can control request bursts.

id="jz4c5k"

Incoming Requests



Token Bucket



Allowed Rejected/Delayed

Requests Requests

This prevents an uncontrolled traffic spike from immediately overwhelming downstream services.

5. Caching Strategy

Product information is read frequently but changes relatively infrequently.

Therefore, cache read-heavy data such as:

* Product details

* Categories

* Product images

* Sale information

* Pricing metadata

id="5zqf8v"

Client



API Gateway



Cache



Cache HIT → Return immediately

Cache MISS



Product Database

Important Design Decision:

Inventory availability is not treated as ordinary cache data.

The authoritative inventory state remains in the transactional database.

This prevents stale cache data from becoming the source of truth for stock deductions.

6. Inventory Scalability

Inventory is the most contention-sensitive component.

The critical operation is: `id="qk7f4t"`

UPDATE inventory

SET `available_quantity` = `available_quantity` - 1

WHERE `product_id` = 'PROD-X'

AND `available_quantity` >= 1;

This operation performs the availability check and deduction atomically.

Why?

Suppose:

`id="q8ggwp"`

Available = 1

Customer A → Buy

Customer B → Buy

Both requests arrive simultaneously.

Only one transaction can successfully satisfy:

`available_quantity >= 1`

After the successful update: `id="2j4w9n"`

`Available = 0`

The other request fails.

Therefore: Successful reservations can never exceed available inventory.

7. Hot-Item Contention Management

A flash sale creates a hot key problem because thousands of customers may access the same product.

SALESTORM reduces contention through:

1. API admission control
2. Rate limiting
3. Short inventory transactions
4. Atomic conditional updates
5. Queueing/backpressure where appropriate
6. Horizontal scaling of stateless services
7. Controlled inventory partitioning for extreme scale

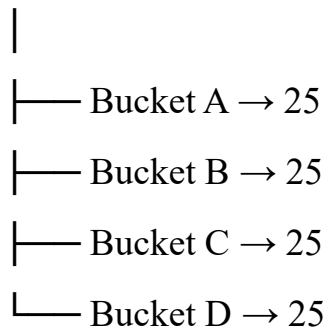
Future scaling approach:

For extremely high demand, inventory can be divided into controlled stock buckets.

`id="a9i7p4"`

Product X

100 Units



Requests can be distributed across buckets while maintaining strict limits on total stock.

This reduces contention on a single database record.

8. Asynchronous Processing

Not every operation needs to happen before the customer receives a purchase result.

The critical path is kept short:

id="6b4a6x"

Request



Inventory Reservation



Payment



Order Confirmation

Non-critical work is asynchronous:

id="2f2m1v"

Order Confirmed



Message Broker



└──→ Shipment

└──→ Analytics

└──→ Audit

Benefits:

- * Reduces customer-facing latency
- * Prevents downstream services from blocking checkout
- * Allows independent scaling
- * Provides buffering during traffic spikes

9. Queue & Backpressure Strategy

The message broker acts as a buffer between services.

id="q0e4hc"

Fast Producer



Message Broker



Controlled Consumers

If a downstream service temporarily processes messages more slowly:

id="k4n9me"

Producer



Queue ↑



Consumer

Messages wait safely instead of immediately overwhelming the consumer.

Monitoring:

The following queue metrics are monitored:

- * Queue depth
- * Consumer lag
- * Processing rate
- * Failed messages
- * Retry count
- * Dead-letter messages

10. Database Scalability

The database is divided conceptually into:

Transactional workload Used for:

- * Inventory
- * Reservations
- * Payments
- * Orders

Read-heavy workload Used for:

- * Product discovery
- * Categories
- * Product information

Read-heavy traffic can use:

id="1ly1iw"

Read Requests



Read Replicas / Cache while critical writes remain directed toward the authoritative transactional database.

11. Database Optimization

SALESTORM uses:

- * Proper indexing
- * Primary keys
- * Foreign keys
- * Unique constraints
- * Short transactions
- * Connection pooling
- * Query optimization
- * Read replicas where appropriate
- * Partitioning where required

Critical indexes include:

id="4x4k3n"

product_id

reservation_id

idempotency_key

payment_id

order_id

expires_at

The `idempotency\_key` should have a unique constraint to prevent duplicate business operations.

12. Reliability Architecture

Reliability means the system should continue operating correctly even when individual components fail.

SALESTORM uses:

id="7z1v2s"

Timeouts

Retries

Circuit Breakers

Idempotency

Transactional Outbox

Message Queues

Dead-Letter Queues

Reservation Expiry

Reconciliation

Health Checks

Monitoring

13. Timeout Strategy

Every external or network-dependent operation has a defined timeout.

Example:

id="3l4x9n"

Payment Service



Payment Gateway



Timeout



Do NOT assume failure immediately



Check / Reconcile Payment Status

A payment timeout does not automatically mean payment failed.

The system first determines the actual payment state before taking a compensating action.

This prevents situations where:

Payment actually succeeded



System assumes failure



Reservation released incorrectly

14. Retry Strategy

Retries are used only for transient failures.

Examples:

- * Temporary network failure
- * Temporary service unavailability
- * Message processing failure

Retries use controlled mechanisms such as:

id="r1r9h7"

Attempt 1



Failure



Backoff



Attempt 2



Backoff



Attempt 3

Exponential backoff prevents the system from repeatedly overwhelming a recovering service.

15. Circuit Breaker

External dependencies such as payment gateways can fail.

SALESTORM protects itself using a Circuit Breaker.

id="j9w5t1"

Payment Gateway



Circuit Breaker



Payment Service

States:

id="x9w8d0"

CLOSED



Repeated failures



OPEN



Recovery timeout

↓

HALF_OPEN

↓

Success → CLOSED

Benefit:

The failing service does not cause cascading failures throughout SALESTORM.

16. Reservation Expiry

Inventory reservations are temporary.

Example:

id="x2z5w1"

100 Available

↓

Customer reserves 1

↓

99 Available

1 Reserved

↓

Payment not completed

↓

Reservation expires

↓

1 Released

↓

100 Available

A background expiry worker identifies expired reservations using:
`expires_at < current_time` and releases the inventory safely.

17. Payment Failure Recovery

Scenario

`id="m7z6l8"`

Reservation Created



Payment Failed



Reservation Released



Inventory Available Again

The customer receives a clear failure response.

No inventory remains permanently blocked.

18. Payment Timeout Recovery

Payment timeout requires special handling.

`id="c4d3p8"`

Payment Request



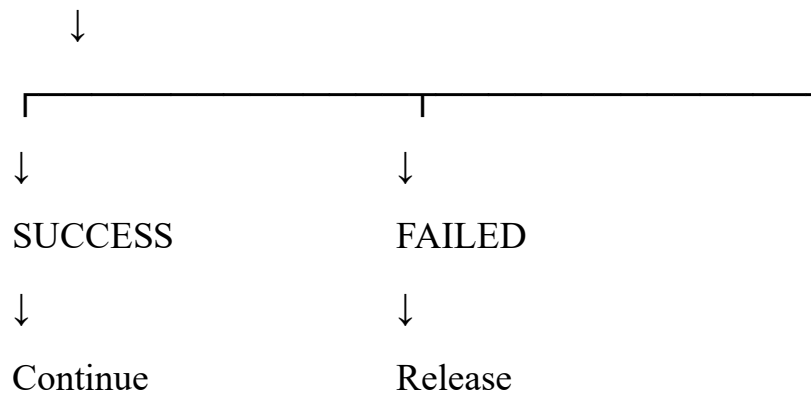
Gateway Timeout



Payment Status Unknown



Reconciliation



This prevents incorrect duplicate payments or premature inventory release.

19. Payment Success + Order Service Failure

This is one of the most important reliability scenarios.

Failure scenario:

id="8j3zq6"

Payment SUCCESS



Order Service DOWN

SALESTORM does not lose the successful transaction.

Recovery:

id="x4y5z7"

Payment Service



Transactional Outbox



PaymentSucceeded Event



Message Broker



Order Service



Retry



Idempotent Processing



Order Confirmed

This ensures that the successful payment eventually results in a valid order.

20. Transactional Outbox

The payment status and event record are stored within the same transaction.

id="2x5s7m"

Database Transaction

|

| — Payment = SUCCESS

|

| — Outbox Event = PaymentSucceeded

A background publisher then sends the event to the message broker.

Benefit:

If the Order Service is unavailable, the event remains durable and can be retried.

21. Dead-Letter Queue

Messages that repeatedly fail processing are moved to a Dead-Letter Queue.

id="f2u9x8"

Message



Consumer



Failure



Retry



Failure



Retry



Failure



Dead-Letter Queue

The operations team can inspect and replay these messages after correcting the underlying problem.

22. Idempotency

Critical operations use idempotency keys.

Examples:

id="k6d4x2"

- ✓ Reservation
- ✓ Payment
- ✓ Order creation
- ✓ Event processing

Example:

id="n4y5t6"

Request 1

Idempotency-Key = ABC123



Payment Created

Request 2

Idempotency-Key = ABC123



Existing Payment Returned

Result:

No duplicate payment and no duplicate order

23. Fault Isolation

Services are designed so that failure of one component does not automatically bring down the entire system.

Example:

id="u2r7e9"

Notification Service DOWN



Order Processing continues



Notification retried later

Similarly:

id="q5w6t1"

Order Service temporarily DOWN



Payment event remains in Outbox/Broker



Order recovered later

This follows the principle:

Non-critical downstream failures should not unnecessarily block the critical purchase path.

24. Health Checks

Each service exposes health information.

Example:

http id="b3n5m8"

GET /health

Possible response:

json id="8y5jv4"

```
{  
  "status": "healthy",  
  "database": "healthy",  
  "message_broker": "healthy",  
  "payment_gateway": "healthy"  
}
```

Health checks allow orchestration infrastructure to remove unhealthy instances from traffic.

25. Observability

SALESTORM monitors three major areas:

Metrics

- * Request rate
- * P50/P95/P99 latency
- * Error rate

- * Inventory reservation success/failure
- * Payment success/failure
- * Order conversion
- * Queue depth
- * Consumer lag
- * Database latency
- * Oversold count

Logs

Critical business events are logged:

id="0z8r6v"

ReservationCreated

ReservationExpired

PaymentInitiated

PaymentSucceeded

PaymentFailed

OrderCreated

OrderReconciled

Distributed Tracing

A single:

id="u7fla3"

X-Correlation-ID can trace:

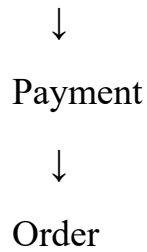
API Gateway

↓

Checkout

↓

Inventory



26. Critical Alerts

The system generates alerts for:

Alert	Reason
-----	-----
Oversold count > 0	Critical inventory integrity violation
High P95/P99 latency	Performance degradation
Payment failure spike	Gateway/system issue
Queue backlog increasing	Consumer capacity problem
Database latency spike	Database bottleneck
Reservation expiry backlog	Inventory may remain blocked
Circuit breaker OPEN	External dependency unhealthy
High error rate	Service degradation

27. Failure Scenario Matrix

Failure	System Response	Expected Result
-----	-----	-----
API instance fails continues	Load balancer routes to another instance	Request
Traffic spike protected	Rate limiting + admission control	Backend

Inventory unavailable	Transaction fails safely	No overselling
Payment fails	Reservation released	Stock restored
Payment timeout	Reconciliation	No duplicate charge
Payment gateway fails isolated	Circuit breaker	Dependency
Order Service fails created	Outbox + broker retry	Order eventually
Message processing fails recovered	Retry	Temporary failure
Repeated message failure preserved	Dead-letter queue	Message
Reservation expires restored	Expiry worker releases stock	Inventory
Database failure	Retry/failover strategy	Service recovery
Duplicate request transaction	Idempotency key	No duplicate

28. 10,000 Concurrent Purchase Scenario

The complete flow is:

id="h9n3x5"

10,000 Customers

|



CDN / WAF

|



Load Balancer



API Gateway



├── Rate Limiting

├── Authentication

└── Admission Control



Checkout Service



Inventory Service



Atomic Conditional UPDATE



├── 100 successful



└── 9,900 rejected



Payment



▼
Transactional Outbox

|
▼
Message Broker

|
▼
Order Service

Expected invariant

`id="5e8p2s"`

Available Stock = 100

Successful Reservations $\leq$ 100

Oversold Count = 0

29. Scalability from 10,000 to 500,000 Requests/sec

The architecture scales in multiple layers.

Layer 1 — Edge CDN + WAF

Absorbs and filters incoming traffic.

Layer 2 — Gateway - API Gateway

Provides rate limiting and admission control.

Layer 3 — Application - Multiple Stateless Service Instances

Scale horizontally.

Layer 4 — Cache

Reduces database reads for product/catalog data.

Layer 5 — Message Broker

Buffers asynchronous workloads.

Layer 6 — Database

Use:

- * Indexing
- * Connection pooling
- * Read replicas
- * Partitioning
- * Controlled inventory buckets

Key principle

Scaling the application layer does not remove inventory contention. Inventory correctness remains protected by the transactional consistency boundary.

30. Scalability vs Consistency Trade-off

SALESTORM deliberately prioritizes inventory consistency over maximum write availability for the critical inventory operation.

For example:

Wrong approach:

Cache says 1 unit available



Two requests accepted



Overselling

SALESTORM instead uses:

Transactional Database



Atomic Conditional Update



Only valid reservations succeed

Trade-off:

The inventory service may reject requests or temporarily become unavailable rather than accepting transactions that could violate stock correctness.

For a limited-stock flash sale, an incorrect successful sale is more damaging than a temporary failed purchase attempt.

31. Reliability Principles

SALESTORM follows these principles:

1. Fail fast rather than fail unpredictably
2. Never assume payment failure after a timeout
3. Never process the same business operation twice
4. Never allow inventory to become negative
5. Persist critical events before asynchronous publishing
6. Retry transient failures with backoff
7. Isolate unhealthy dependencies
8. Release expired inventory automatically
9. Monitor critical business invariants
10. Design recovery paths before failures occur

32. Key Reliability Invariants

The system maintains the following invariants:

Inventory:

`id="a7f8k2"`

`available_quantity >= 0`

Successful sales:

`id="d2j4m8"`

`sold_quantity <= original_stock`

Reservation:

`id="p6r5t9"`

Active reservations must have valid expiry times.

Payment:

`id="n8x3c1"`

One idempotency key → One payment transaction

Order:

`id="z4q7w6"`

One successful purchase → At most one confirmed order

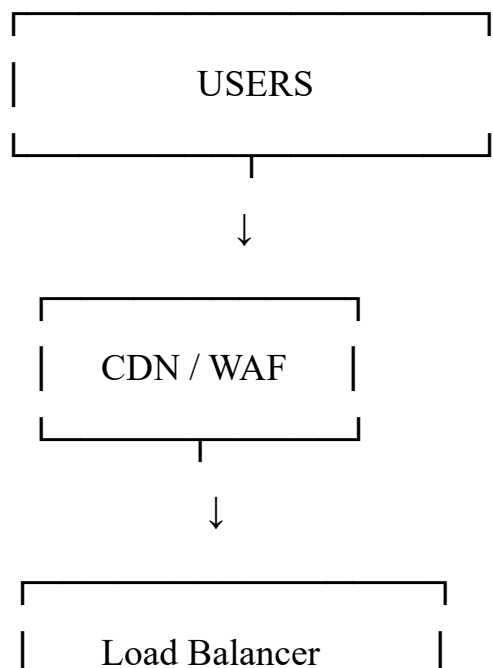
Core Business Guarantee:

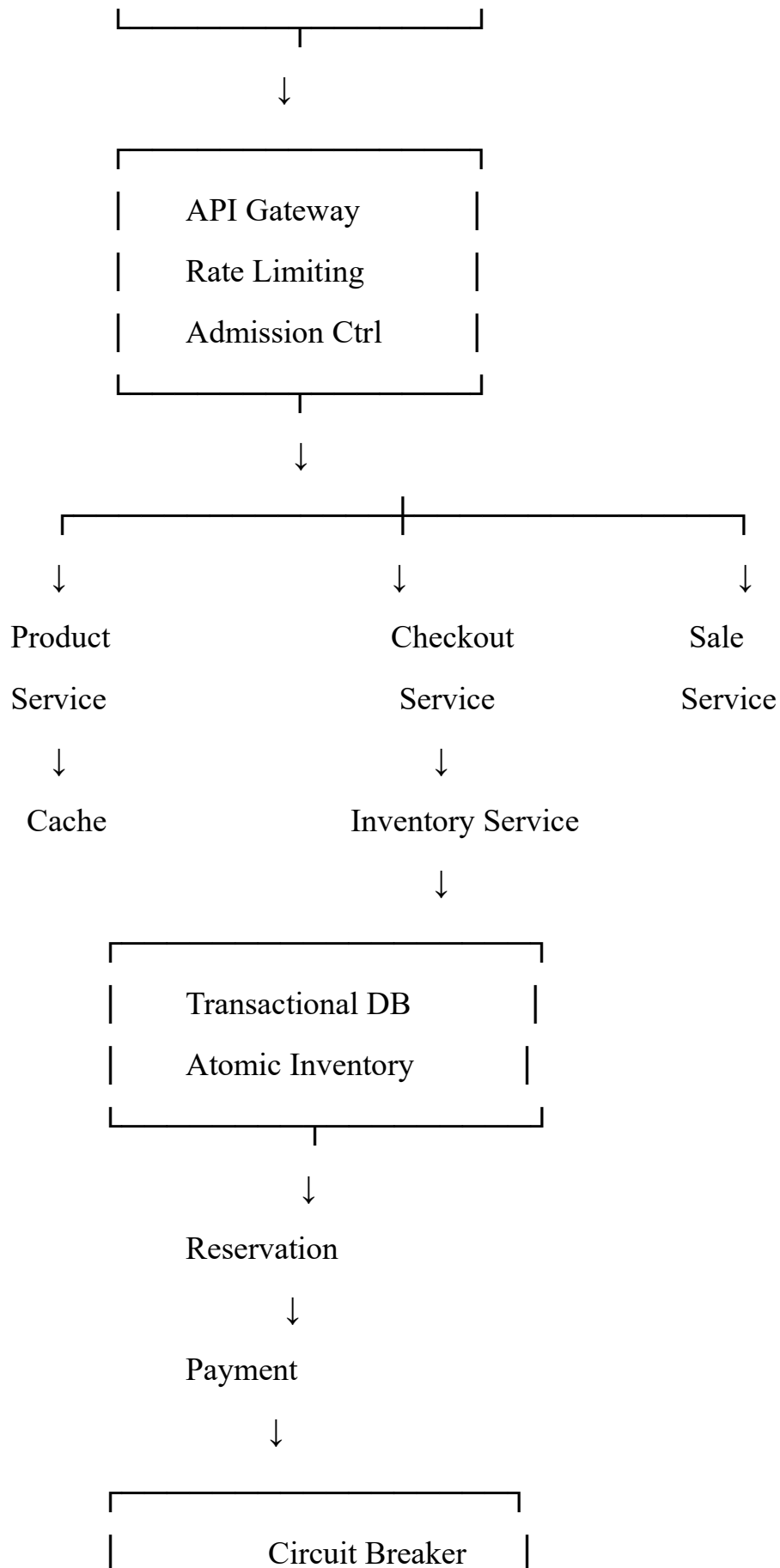
`id="e5m8v2"`

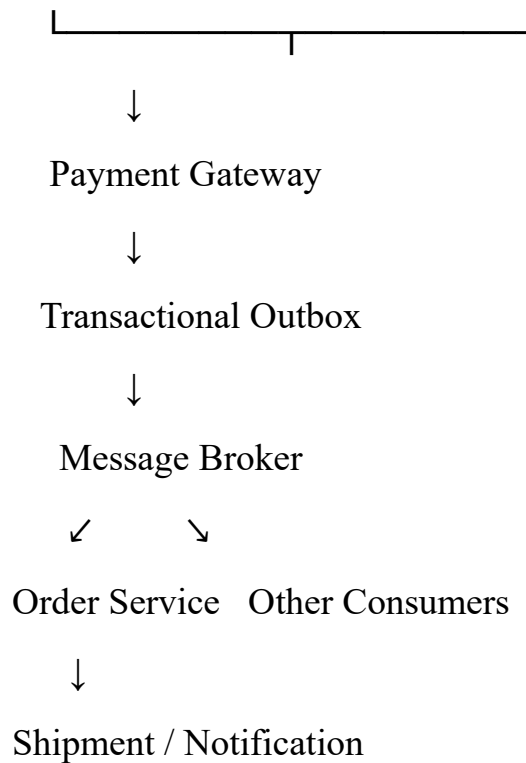
`OVERSOLD_COUNT = 0`

33. Final Scalability & Reliability Architecture

`id="v8c4n2"`







34. Final Design Statement

SALESTORM achieves scalability through horizontal scaling, caching, admission control, queueing and database optimization, while reliability is achieved through atomic inventory transactions, idempotency, timeouts, retries, circuit breakers, transactional outbox, message-based recovery and reservation expiry.

The architecture deliberately protects the inventory consistency boundary even under extreme concurrency. Therefore, whether the system receives 10,000 concurrent purchase attempts or scales toward 500,000 requests/sec, the fundamental business invariant remains unchanged: successful sales can never exceed the available inventory, and `OVERSOLD = 0`.